Malware Code Injection

Joe Wu

Agenda

- Overview of malware code injection
- Different kinds of code injection technique and their usage in malware
- What do investigators look at in code injections
- How to detect

Overview

- When we run malware such as Dridex payload, Brazilian malware in the lab, nothing seems happen. Actually, behind view, there is code injection took place.

In this presentation,

- we will review the basic idea of code injection
- see some examples of malware code injection
- known why it is stealthy
- What do investigators look at in code injections
- Learn different kinds of code injections and their usage in malware
  - DLL injection
  - Process replacement (RunPE)
  - Hook injection
  - APC injection

Examples of code injection

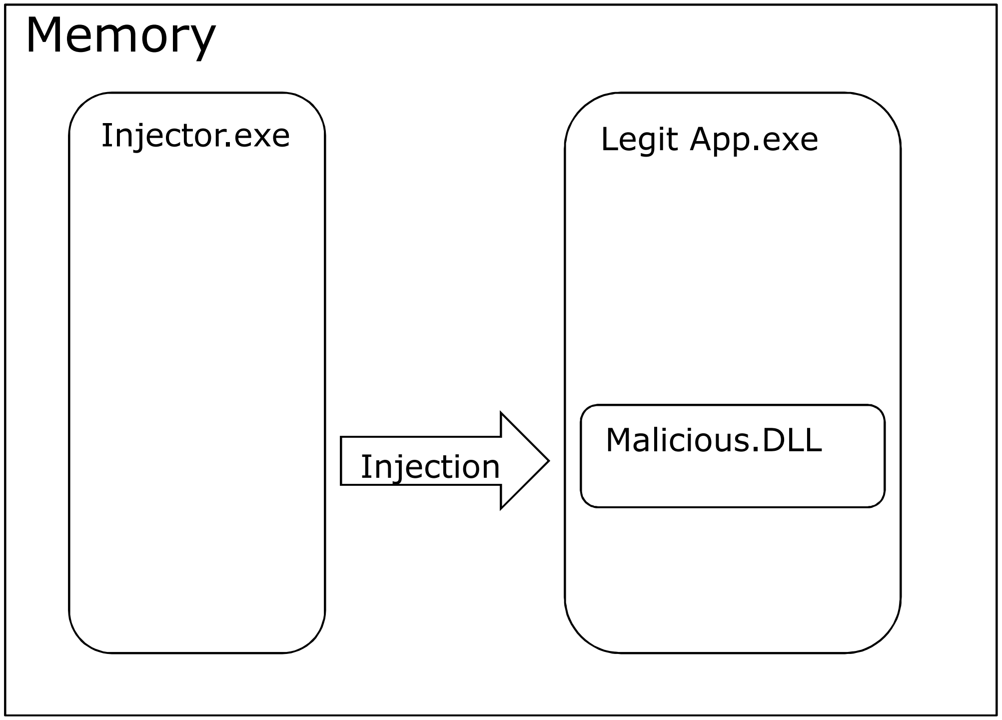
- CASE0541 - .Net IL injector
- CASE9208 – Brazilian malware (RunPE malware)
- CASE0353 – Powershell Spam
- CASE0194 – Dridex payload
- Bangladesh bank got stolen $8M in Swift bank payment system
- Independent penetration tester announced to “audit” bank software

- DLL Injection

Dll Injection

- DLL injection is a way that malware can run a DLL inside another process, therefore providing that DLL with all the privileges of the target process.
- It is a common way to force a process to load malicious code.

- Steps:



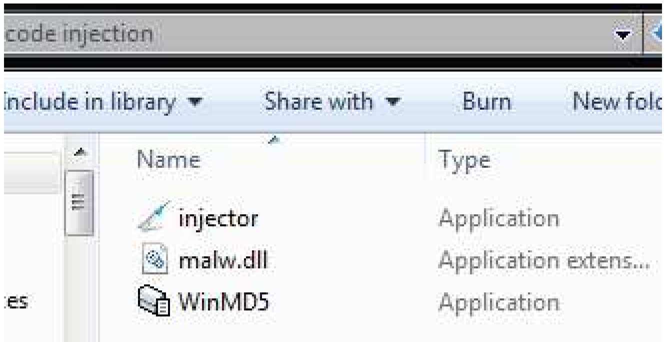
Dll Injection 2 - Demo

- 3 files involved:

  Injector.exe – malware injector

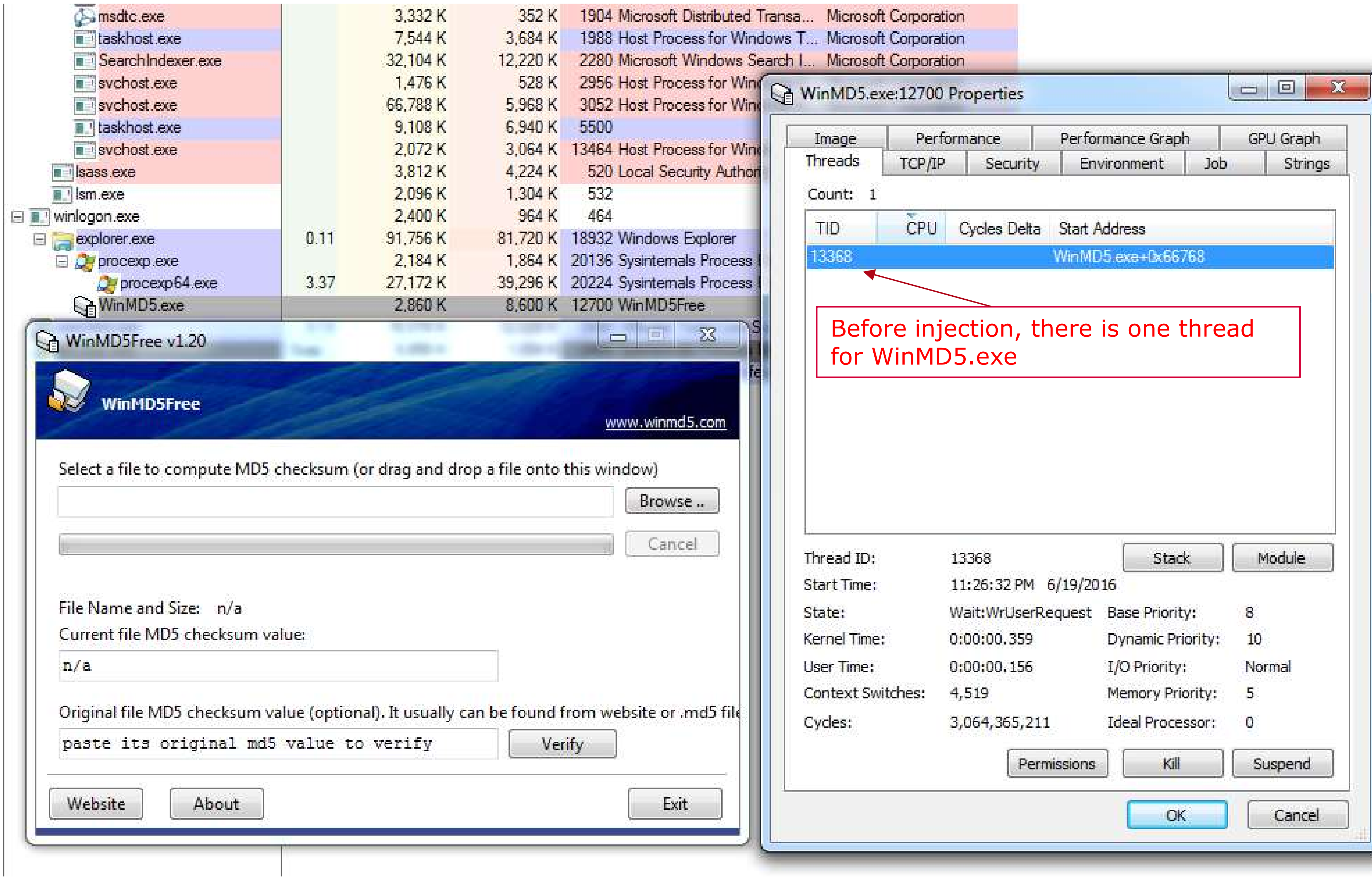
  Malw.dll - malicious DLL

  WinMD5.exe – legit app

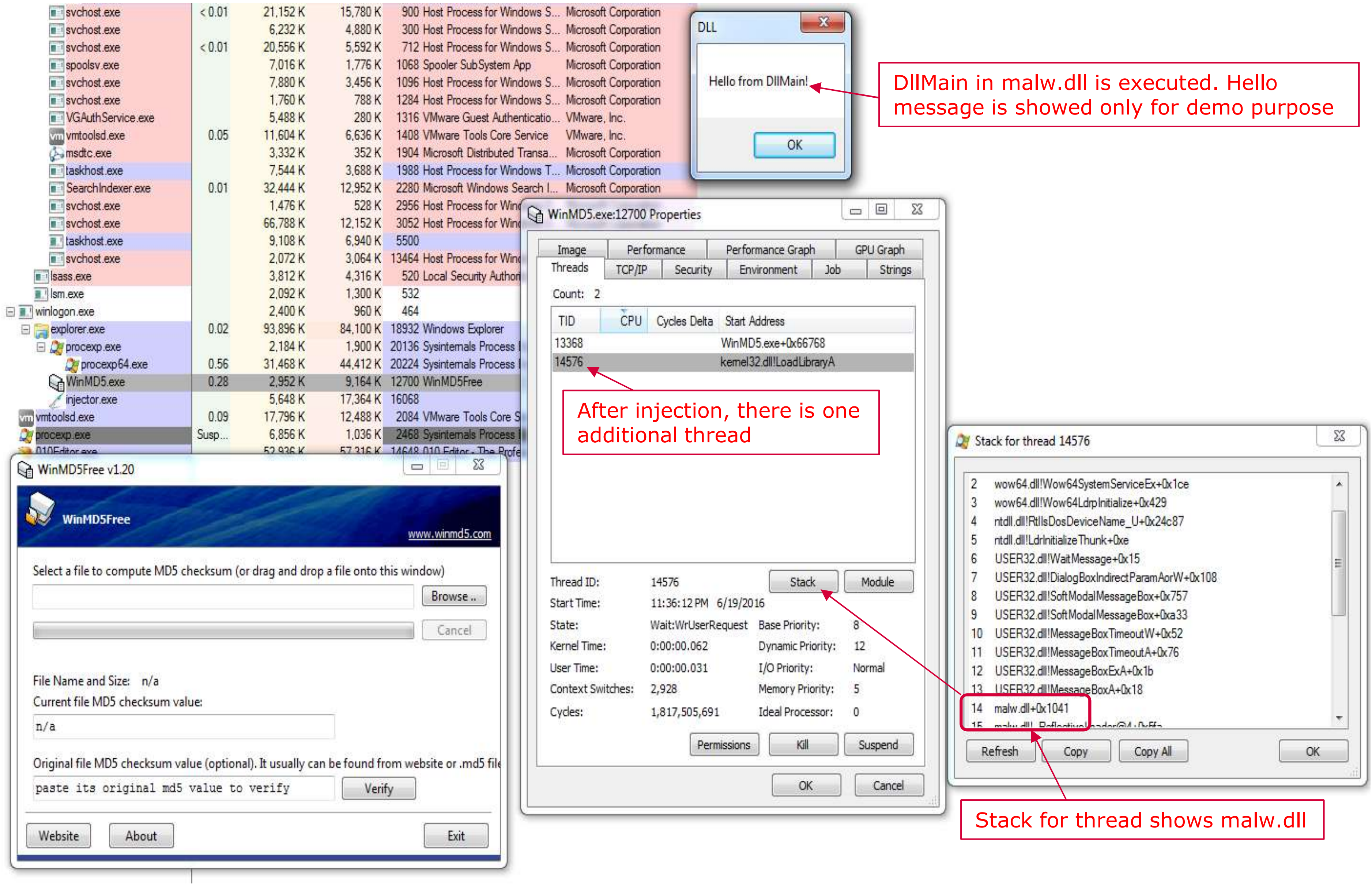


- Legit app is running in memory (WinMD5.exe)
- We run malware injector.exe with parameter “Malw.dll”, “WinMD5.exe”.
- Result: Malw.dll is injected to WinMD5.exe and starts execution.

Dll Injection 3 - Demo before dll injection



Dll Injection 4 – demo after dll injection

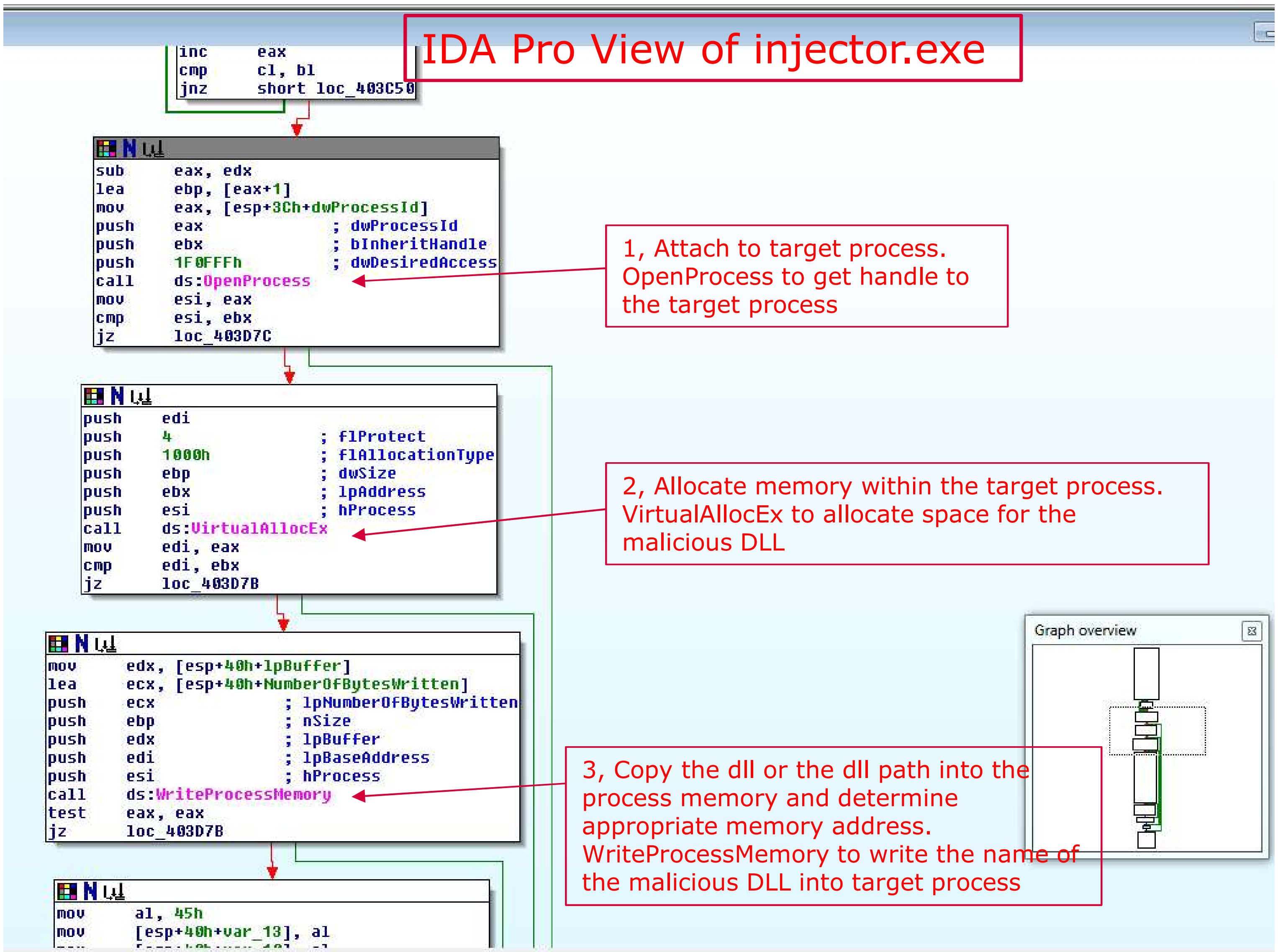


Dll Injection 5

- DLL Injection calls APIs
- Pseudocode of Injector.exe:

```
hTargetProcess = OpenProcess(,,TargetProcessID);
pNameProc=VirtualAllocEx(hTargetProcess,,,)
WriteProcessMemory(hTargetProcess,,maliciousDLLName,,,)
GetModule("Kernel32.dll")
GetProcAddress(..,"LoadLibraryA");
CreateRemoteThread(hTargetProcessm,,,,pNameProc,,)
```

DLL Injection 6



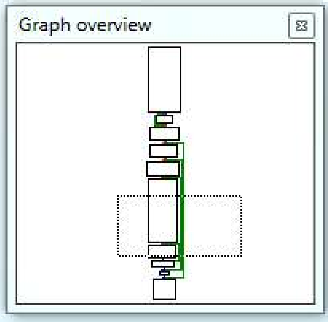
DLL Injection 7

IDA Pro View of injector.exe

```
lea     eax, [esp+40h+ThreadId]
push    eax             ; lpThreadId
mov     cl, 61h
push    ebx             ; dwCreationFlags
mov     [esp+48h+var_22], cl
mov     [esp+48h+var_1C], cl
push    edi             ; lpParameter
lea     ecx, [esp+4Ch+ProcName]
push    ecx             ; lpProcName
lea     edx, [esp+50h+ModuleName]
push    edx             ; lpModuleName
mov     [esp+54h+ModuleName], 4Bh
mov     [esp+54h+var_12], 52h
mov     [esp+54h+var_11], 4Eh
mov     [esp+54h+var_E], 33h
mov     [esp+54h+var_D], 32h
mov     [esp+54h+var_C], 2Eh
mov     [esp+54h+var_B], 44h
mov     [esp+54h+var_8], bl
mov     [esp+54h+var_23], 6Fh
mov     [esp+54h+var_21], 64h
mov     [esp+54h+var_1F], 69h
mov     [esp+54h+var_1E], 62h
mov     [esp+54h+var_1A], 79h
mov     [esp+54h+var_19], 41h
mov     [esp+54h+var_18], bl
call    ds:GetModuleHandleA
push    eax             ; hModule
call    ds:GetProcAddress
push    eax             ; lpStartAddress
push    ebx             ; dwStackSize
push    ebx             ; lpThreadAttributes
push    esi             ; hProcess
call    ds:CreateRemoteThread
mov     ebp, eax
cmp     ebp, ebx
jz      short loc_403D74
```

```
push    0FFFFFFFFh      ; dwMilliseconds
push    ebp             ; hHandle
call    ds:WaitForSingleObject
push    8000h           ; dwFreeType
push    ebx             ; dwSize
push    edi             ; lpAddress
push    esi             ; hProcess
call    ds:VirtualFreeEx
mov     ebx, 1
```

4, Instruct the process to execute the dll by calling CreateRemoteThread, and pass the control to DllMain



DLL Injection 8

- In Dll injection, the malware injector never calls a malicious function. The DllMain in the dll is called automatically the dll is loaded into memory.
- The goal of dll injection injector is to call CreateRemoteThread so that a thread is created in the target process.

- Once we find the Dll injection activity in disassembly, we should start looking for strings containing the names of the malicious DLL and the victim process.
- the starter normally get victim PID by comparing the process's name, for this case, it is WinMD5.exe
- To find the malicious dll name, we could set a breakpoint at WriteProcessMemory, and check the stack memory to find the value of Buffer that passed to WriteProcessMemory.

- Process Replacement

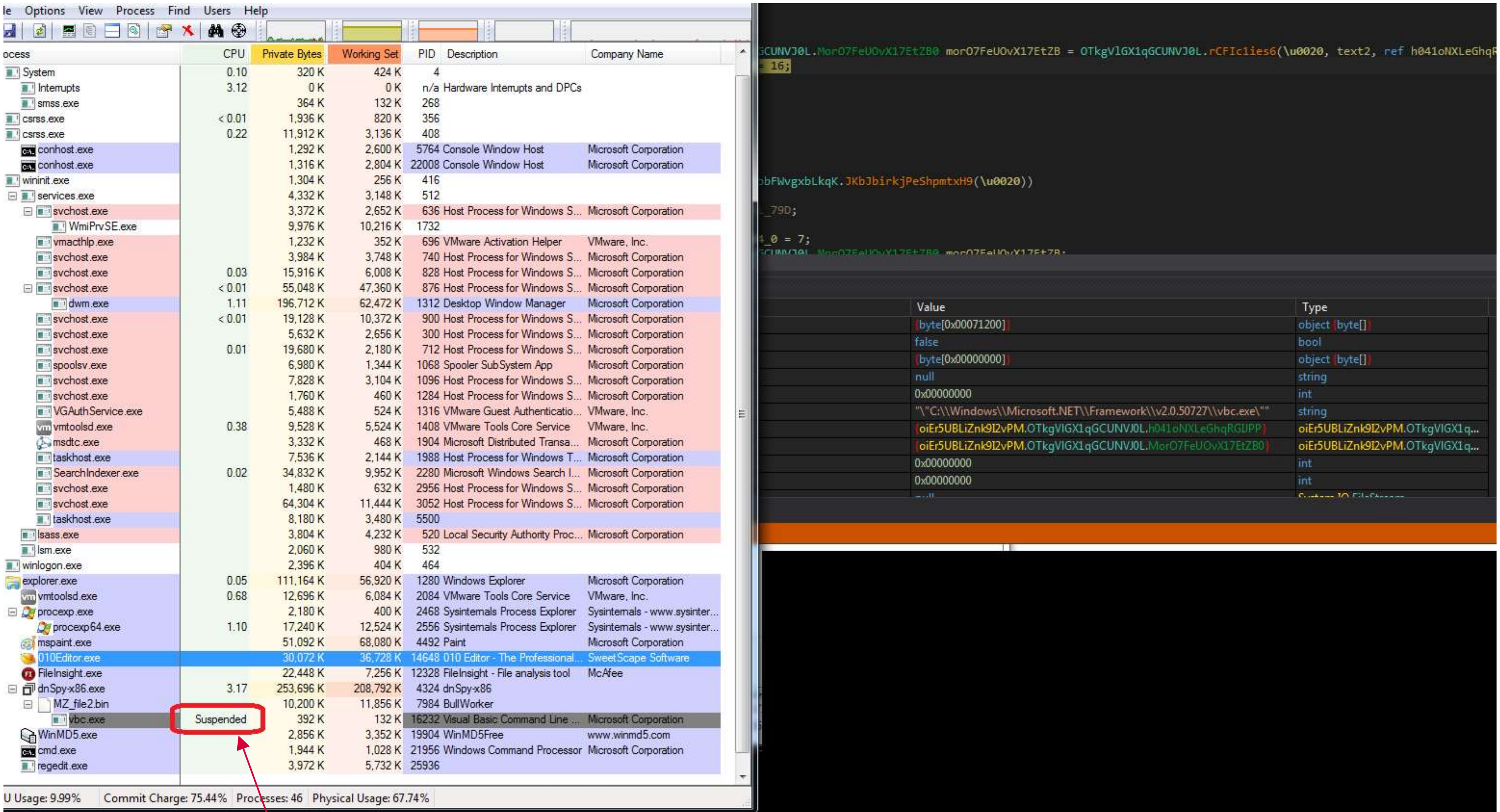
Process replacement (RunPE)

- Malware load a process into memory with a SUSPENDED state.
- Replace the process's memory with malicious executable.
- Resume the process to run the malicious executable

- Malware appears non-malicious by disguising as the normal windows process.
- Bypass firewall, IPS.
- Example: CASE9208, CASE0541
- Key API calls:
  - ZwUnmapViewOfSection
  - ResumeThread

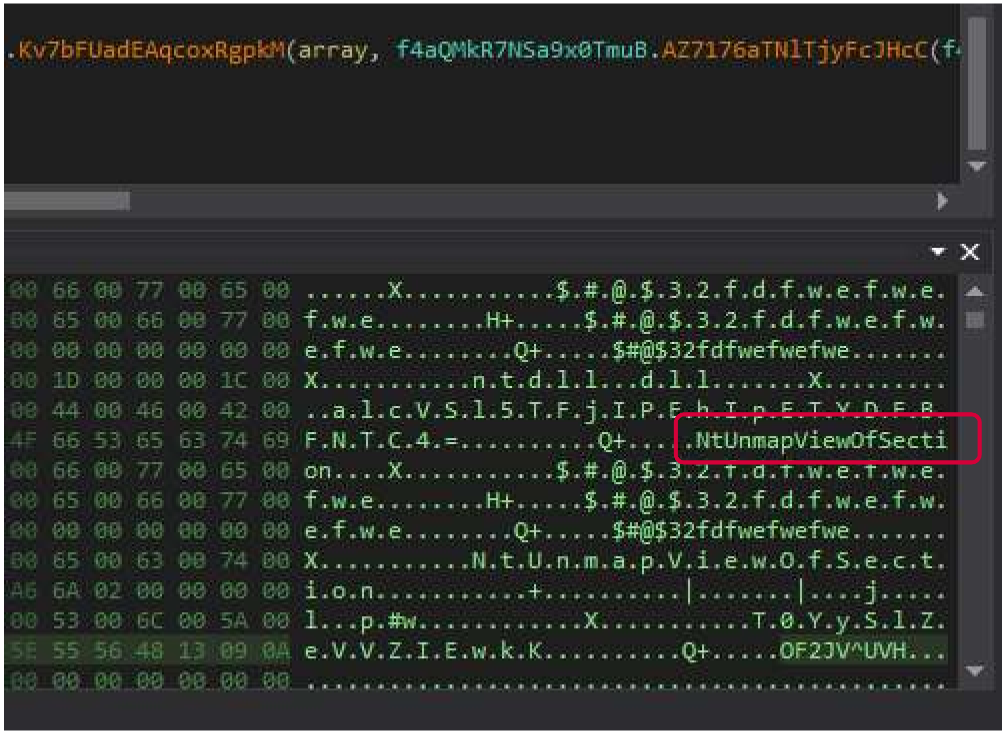
- Detection: compare the PE header of memory with the legit file on disk

Process replacement (RunPE)



Malware loads process in Suspended status
C:\Windows\Microsoft.Net\Framework\v2.0.50727\vbc.exe

Process replacement (RunPE)



Malware calls NtUnmapViewOfSection to replace the memory of legit process with malicious code

Hook Injections

- Hook Injection intercepts messages destined for applications.
- SetWindowsHookEx is used to intercept Windows messages
- Often seen in Keyloggers

APC Injections

- APC (Asynchronous procedure call) is OS feature that allows user application and OS components to execute code in a thread. APCs are processed when a thread calls functions like Sleep, WaitForSingleObjectEx.
- Investigators look for QueueUserAPC API calls
- Often seen in RootKit and malware drivers

What do Malware analysts look at code injection

- Malware analysts identify injection techniques in order to find malware on a live system.
- Find the target process by tracing the malware calling to APIs. Breakpoints may be needed at APIs.
- Find the malicious DLL by identify important strings in memory.

Impacts to security controls - bypassing

- Code injection is stealthy and dangerous.
- By injecting to legit IE, malware could gain communicate to C2 server, bypassing Firewall
- By injecting to legit application in the memory, malware could bypass hash based AV detection, Bit9, or IPS.
- By injecting to Windows process, malware could gain system privilege
- Malware could hide from viewing in a process list

Detection and remediation

- Review API calls
- Compare PE file in memory and on disk
- Memory forensic, volatility

Malware code injection

- Questions

- Thank you!